



Heavily based on material from:
Murach's MySQL (3rd Edition), by Joel Murach

Objectives

Applied

1. Given a set of SQL statements to be combined into a **transaction**, write a script that begins, commits, and rolls back the transaction.

Knowledge

1. Describe the use of **transactions**.
2. Describe the way **locking helps prevent concurrency problems**.
3. Describe the use of **save points**.
4. Describe the way the **transaction isolation level** affects concurrency problems and performance.
5. Describe the **options for locking** selected rows and how they can prevent concurrency problems.
6. Describe a **deadlock**.
7. Describe **three techniques that can reduce deadlocks**.

Transaction

A **transaction** represents **a unit of work** performed within a database management system against a database, and **treated in a coherent and reliable way independent of other transactions**. A transaction ***generally* represents any change in database**.

Transactions in a database environment have two main purposes:

1. To provide **reliable units of work** that **allow correct recovery from failures** and **keep a database consistent** even in cases of system failure, when execution stops (completely or partially) and many operations upon a database remain incomplete, with unclear status.
2. To **provide isolation** between programs accessing a database concurrently. If this isolation is not provided, the programs' outcomes are possibly erroneous.

ACID

Atomicity, **C**onsistency, **I**solation, **D**urability, is a set of properties that guarantee that database transactions are processed reliably.

In the context of databases, a **single logical operation** on the **data is called a transaction**.

- For example, a transfer of funds from one bank account to another, even involving multiple changes such as debiting one account and crediting another, is a single transaction.



ACID

Jim Gray defined the properties of a reliable transaction system in the late 1970s and developed technologies to achieve them automatically.

In 1983, Andreas Reuter and Theo Härder coined the acronym **ACID** to describe them.



ACID - Atomicity

Atomicity requires that each transaction be "**all or nothing**": if one part of the transaction fails, the entire transaction fails, and the database state is left unchanged. When any portion of a transaction fails, the entire transaction is rolled back (undone).

- An atomic system must guarantee atomicity in each and every situation, including power failures, errors, and crashes.
- To the outside world, a committed transaction appears (by its effects on the database) to be indivisible ("**atomic**"), and an aborted transaction does not happen.
- In 2009 Samoa switched from driving on the right side of the road to the left. That is a transaction that you really need to happen all at once.

ACID - Consistency

The consistency property ensures that any transaction will bring the database from one valid state to another.

- Any data written to the database must be valid according to all defined rules, including constraints, cascades, triggers, and any combination thereof.
- This does not guarantee correctness of the transaction in the ways the application programmer might have wanted, but merely that programming errors cannot result in the violation of database defined rules.

ACID - Isolation

The isolation property ensures that the concurrent execution of transactions results in a system state that would be obtained if transactions were executed serially, i.e., one after the other.

- Providing isolation is the main goal of concurrency control.
- Depending on the concurrency control method (i.e. if it uses strict – as opposed to relaxed – serializability), the effects of an incomplete transaction might not even be visible to another transaction.
- What could possibly go wrong if we act on tentative (uncommitted) information?

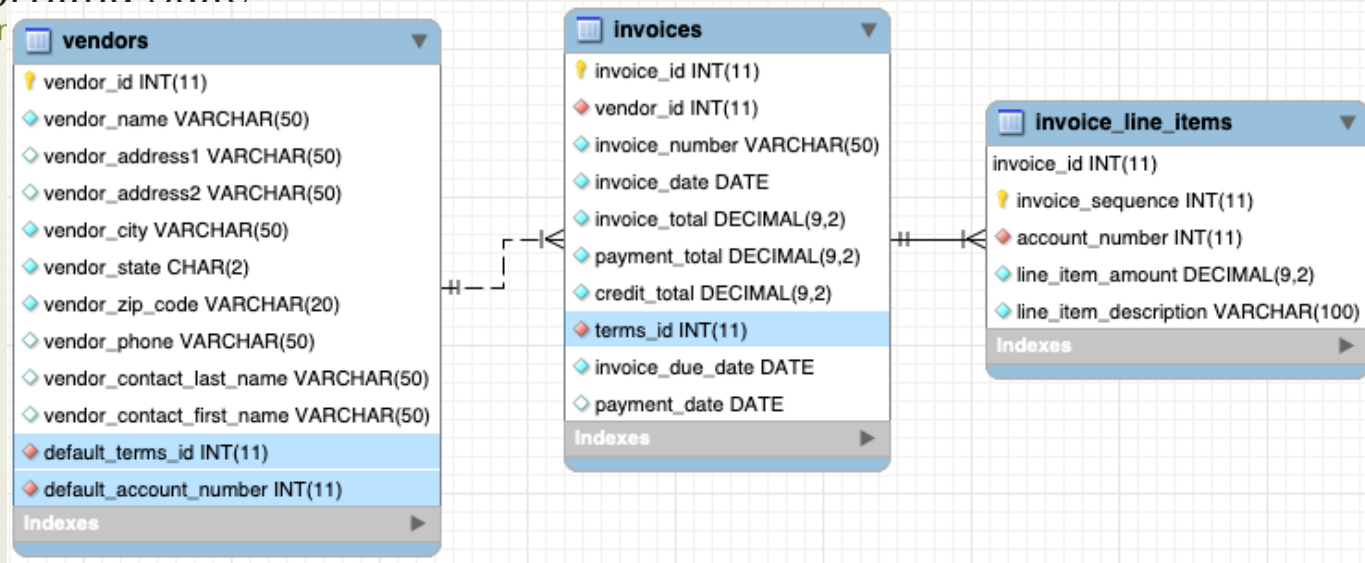
ACID - Durability

The durability property ensures that once a transaction has been committed, it will remain so, even in the event of power loss, crashes, or errors.

- Once a group of SQL statements execute, the results need to be stored permanently (even if the database crashes immediately thereafter).
- To defend against power loss, transactions (or their effects) must be recorded in a **non-volatile memory**.
- Ever have your computer crash before you've saved that important Word document?

When to use transactions

- When you code **two or more INSERT, UPDATE, or DELETE statements that affect related data** (parent and child relationships).
- When you **move rows from one table to another table** by using **INSERT** and **DELETE** statements. Don't delete the data from the existing table unless it has been copied into the new table.
- **When failure of an INSERT, UPDATE, or DELETE statement would violate data integrity.**
- A transaction can either complete successfully with a **COMMIT** or it can **ROLLBACK**.
- A **COMMIT** makes the transaction complete and durable.
- A **ROLLBACK** returns the database to the state it was in before the transaction began.



We insert a new row into the **invoices** table. It is a new invoice for an existing vendor.

We insert a row into the **invoice_line_items** table, for the invoice.

When we insert a second row into the **invoice_line_items** table, the insert fails. The **line_item_description** is invalid.

We do not want to have a partial invoice in the database, so we roll back the entire transaction. **We want to have the entire transaction or nothing.**

Later, we fix the **line_item_description** and can insert the entire invoice.

Two transactions that retrieve and then modify the data in the same row

Transaction A

START TRANSACTION;

```
UPDATE invoices SET credit_total = credit_total + 100  
WHERE invoice_id = 6;
```

```
-- the SELECT statement in Transaction B  
--      won't show the updated data  
-- the UPDATE statement in Transaction B  
--      will wait for transaction A to finish
```

COMMIT;

```
-- the SELECT statement in Transaction B  
--      will display the updated data  
-- the UPDATE statement in Transaction B  
--      will execute immediately
```

Two transactions that retrieve and then modify the data in the same row (continued)

Transaction B

START TRANSACTION;

```
SELECT invoice_id, credit_total  
FROM invoices WHERE invoice_id = 6;
```

```
UPDATE invoices SET credit_total = credit_total + 200  
WHERE invoice_id = 6;
```

COMMIT;

How to test these transactions

- Open a separate connection for each transaction.
- Execute one statement at a time, alternating between the two transactions.

```
select * from books2.authors ;
```

```
START TRANSACTION;
```

```
update books2.authors set  
    author = 'R. Jesse Chaney'  
where author = 'Jesse Chaney';  
commit;
```

```
START TRANSACTION;
```

```
update books2.authors set  
    author = 'Raymond Jesse Chaney'  
where author = 'Jesse Chaney';  
commit;
```

```
update books2.authors set  
author = 'Jesse Chaney'  
where author = 'R. Jesse Chaney';
```

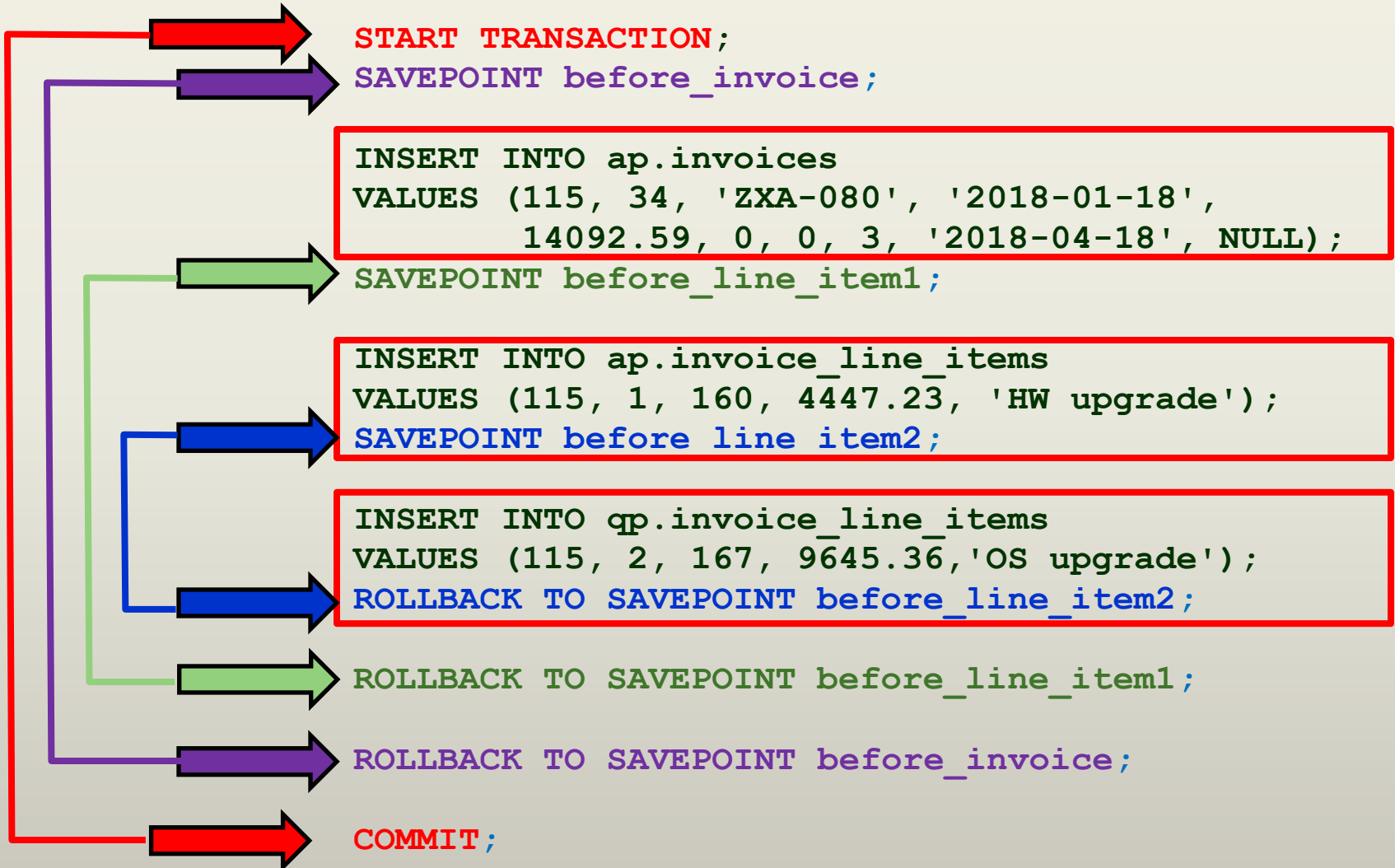
Savepoint

The PostgreSQL storage engine supports the SQL statements **SAVEPOINT**, **ROLLBACK TO SAVEPOINT**, and **RELEASE SAVEPOINT**.

The **SAVEPOINT** statement sets a named transaction savepoint with a name of the identifier. If the current transaction has a savepoint with the same name, the old savepoint is deleted and a new one is set.

The **ROLLBACK TO SAVEPOINT** statement **rolls back a transaction to the named savepoint without terminating the transaction**.

A script that uses save points



Concurrency and Locking

Concurrency is simply supporting 2 or more **transactions** working on the same data at the same time.

Concurrency is **only** a problem when data are being modified by **multiple transactions**.

You avoid database concurrency **problems** by using **locks**.

A lock will delay (or prevent) a transaction if it conflicts with a transaction that is already running.

The four types of concurrency problems that locking can prevent

1. Lost updates
2. Dirty reads
3. Nonrepeatable reads
4. Phantom reads

The four types of concurrency problems

Problem	Description
Lost updates	Occur when two transactions select the same row and then update the row based on the values originally selected.
Dirty reads (uncommitted dependencies)	Occur when a transaction selects data that isn't committed by another transaction.
Nonrepeatable reads (inconsistent analysis)	Occur when two SELECT statements of the same data result in different values because another transaction has updated the data in the time between the two statements.
Phantom reads	Occur when you perform an update or delete on a set of rows when another transaction is performing an insert or delete that affects one or more rows in that same set of rows.

The four types of concurrency problems, #1

Lost Updates

Lost updates occur when two or more transactions select the same row and then update the row based on the value originally selected. Each transaction is unaware of other transactions. The last update overwrites updates made by the other transactions, which results in lost data.

For example, two editors make an electronic copy of the same document.

- Each editor changes the copy independently and then saves the changed copy, thereby overwriting the original document.
- The editor who saves the changed copy last overwrites changes made by the first editor.
- This problem could be avoided if the second editor could not make changes until the first editor had finished.

The four types of concurrency problems, #2

Dirty Read (Uncommitted Dependency)

Dirty Read occurs when a second transaction selects a row that is being updated by another transaction. The second transaction is reading data that has not been committed yet and may be changed by the transaction updating the row.

For example, an editor is making changes to an electronic document.

- During the changes, a second editor takes a copy of the document that includes all the changes made so far, and distributes the document to the intended audience.
- The first editor then decides the changes made so far are wrong and removes the edits and saves the document.
- The distributed document contains edits that no longer exist, and should be treated as if they never existed.
- This problem could be avoided if no one could read the changed document after the first editor determined that the changes were final.



The four types of concurrency problems, #3

Non-repeatable Read

A non-repeatable Read occurs when a second transaction accesses the same row several times and reads different data each time. This is similar to uncommitted dependency in that another transaction is changing the data that a second transaction is reading. However, in non-repeatable read, the data read by the second transaction was committed by the first transaction. Also, non-repeatable read involves multiple reads (two or more) of the same row and each time the information is changed by another transaction; thus, the term non-repeatable read.

An editor reads the same document twice, but between each reading, the writer rewrites the document.

- When the editor reads the document for the second time, it has changed.
- The original read was not repeatable.
- This problem could be avoided if the editor could read the document only after the writer has finished writing it.

The four types of concurrency problems, #4

Phantom Reads

Phantom reads occur when an insert or delete action is performed against a row that belongs to a range of rows being read by a transaction.

The transaction's first read of the range of rows shows a row that no longer exists in the second or succeeding read, as a result of a deletion by a different transaction.

Similarly, as the result of an insert by a different transaction, the transaction's second or succeeding read shows a row that did not exist in the original read.

- **A phantom read occurs** when, in the course of a transaction, two identical queries are executed, and the collection of rows returned by the second query is different from the first.

How does PostgreSQL handle all these?

Isolation Level

The letter **I** in **ACID** stands for **isolation** and essentially symbolizes that **all transactions should run in complete ignorance of each other**. This isolation refers to writes but in particular also to reads of data which might have been changed by other, concurrently ongoing transactions.

- **User A should not read the changes made by user B until they are committed**. Otherwise, user A would read state that is pending for further overwrites or even possible rollback. Such reads are known as Dirty Reads and are a **very undesirable behavior in almost every case**.
- Only one of the isolation level options can be set at a time and it remains set for that **connection** until it is explicitly changed.

The concurrency problems prevented by each transaction isolation level

Isolation level

Problems prevented

“Allowed,” but treated as read committed.

READ UNCOMMITTED

Not allowed in PostgreSQL

READ COMMITTED

Dirty reads

REPEATABLE READ

Dirty reads, lost updates, nonrepeatable reads

SERIALIZABLE

All

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read	Serialization Anomaly
Read uncommitted	Allowed, but not in PostgreSQL	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Allowed, but not in PostgreSQL	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

The syntax of the SET TRANSACTION ISOLATION LEVEL statement

```
SET TRANSACTION ISOLATION LEVEL  
    {READ UNCOMMITTED|READ COMMITTED|  
     REPEATABLE READ|SERIALIZABLE} ;
```

Set the level to SERIALIZABLE for the next transaction

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Isolation Levels, #1

Read Uncommitted (aka dirty read)

A transaction T1 executing under this isolation level can access data changed by concurrent transaction(s). For example, if a concurrent transaction T2 updates a row R1, it can still be read under T1 even though T2 can potentially roll back later.

- Pros: No read locks needed to read data (i.e. no reader/writer blocking). Note, T1 still takes transaction duration locks for any data modified.
- Cons: Data are not guaranteed to be transactionally consistent!
- Usage: It is typically used in queries/applications where data inconsistency can be tolerated.
 - For example, computing average salary of employees.

Isolation Levels, #2

Read Committed

A transaction T1 executing under this isolation level can only access committed data. For example, if a concurrent transaction T2 updates a row R1, it cannot be accessed under T1, in fact T1 will get blocked until T2 either commits or rolls back.

- Pros: Good compromise between concurrency and consistency.
- Cons: Locking and blocking. The data can change when accessed multiple times within the same transaction.
- Usage: Very commonly used isolation level.
- **This is the default isolation level for PostgreSQL.**

Isolation Levels, #3

Repeatable Read

A transaction T1 executing under this isolation level can only access committed data with an additional guarantee that any data read cannot change (i.e. it is repeatable) for the duration of the transaction.

- Pros: **Higher data consistency.**
- Cons: **Locking and blocking.** The locks are held for the duration of the transaction that can lower the concurrency. It does not protect against phantom rows.

Isolation Levels, #4

Serializable

A transaction executing under this isolation level provides the **highest data consistency** including elimination of phantoms but at the cost of **reduced concurrency**. It prevents phantoms by taking a range lock or table level lock if range lock can't be acquired (i.e. no index on the predicate column) for the duration of the transaction.

- Pros: **Full data consistency** including phantom protection. Serializable isolation level guarantees transactions will end up with one possible serial order with an appearance that concurrent transactions did not interfere with each other.
- Cons: **Locking and blocking**. The locks are held for the duration of the transaction that can lower the concurrency.
- Usage: It is used in cases where data consistency is an absolute requirement.

Why not always use Serializable?

The question arises why PostgreSQL, MS SQL Server, and Oracle (and others), NOT set **Serializable** as default isolation level?

The typical answer is: complexity of the implementation which imposes a certain amount of overhead **and possibly poor performance.**

Essentially it boils down to real locking, as well as discovering and preventing deadlocks.

Oh yes, and performance.

Four transactions that show how to work with locking reads

In some cases, the default isolation level of REPEATABLE READ doesn't work the way you want.

Transaction A

```
-- lock row with rep_id of 2 in parent table  
SELECT * FROM sales_reps WHERE rep_id = 2 FOR SHARE;
```

```
-- Transaction B waits for transaction A to finish  
-- Transaction C returns an error immediately  
-- Transaction D skips the locked row and returns  
-- the other rows immediately
```

```
-- insert row with rep_id of 2 into child table  
INSERT INTO sales_totals  
    (rep_id, sales_year, sales_total)  
VALUES (2, 2019, 138193.69);
```

```
COMMIT; -- Transaction B executes now
```

- You can add a **FOR SHARE** clause to the end of a **SELECT** statement.
- This locks the selected rows so other transactions can read them but can't modify them until your transaction commits.

Four transactions that show how to work with locking reads (continued)

Transaction B

```
START TRANSACTION;  
SELECT * FROM sales_reps WHERE rep_id < 5 FOR UPDATE;  
COMMIT;
```

Transaction C

```
START TRANSACTION;  
SELECT * FROM sales_reps WHERE rep_id < 5  
FOR UPDATE NOWAIT;  
COMMIT;
```

A locking read that uses NOWAIT never waits to acquire a row lock. The query executes immediately, failing with an error if a requested row is locked.

You can add a FOR UPDATE clause to the end of a SELECT statement.

- This locks the selected rows and any associated indexes just like an UPDATE statement does.
- Then, other transactions can't read or modify these rows until your transaction commits.

Four transactions that show how to work with locking reads (continued)

Transaction D

```
START TRANSACTION;  
SELECT * FROM sales_reps WHERE rep_id < 5  
    FOR UPDATE SKIP LOCKED;  
COMMIT;
```

This is often done in cursors.

A locking read that uses SKIP LOCKED never waits to acquire a row lock. The query executes immediately, removing locked rows from the result set.

How to test the transactions for locking reads

- Open a separate connection for transaction A and one of the other transactions.
- Execute one statement at a time, alternating between the two transactions.

UPDATE statements that illustrate deadlocking

Transaction A

```
START TRANSACTION;  
UPDATE savings SET balance = balance - transfer_amount;  
UPDATE checking SET balance = balance + transfer_amount;  
COMMIT;
```

Transaction B (possible deadlock)

```
START TRANSACTION;  
UPDATE checking SET balance = balance - transfer_amount;  
UPDATE savings SET balance = balance + transfer_amount;  
COMMIT;
```

Transaction B (prevents deadlocks)

```
START TRANSACTION;  
UPDATE savings SET balance = balance + transfer_amount;  
UPDATE checking SET balance = balance - transfer_amount;  
COMMIT;
```

How to prevent deadlocks

- Don't allow transactions to remain open for very long.
- Don't use a transaction isolation level higher than necessary.
- Make large changes when you can be assured of nearly exclusive access.
- Consider locking when coding your transactions.





Why do we have all these locking modes?

Deadlocking

- A deadlock occurs when there is a **cyclic dependency** between two or more threads/transaction for some set of resources.
- Deadlock is a condition that can occur on any system with concurrent resource access, not just on a relational database management system.
 - A thread in a multi-threaded system may acquire one or more resources (for example, **locks**).
 - If the resource being acquired is currently owned by another thread, the first thread may have to wait for the owning thread to release the target resource.
 - The waiting thread is said to have a dependency on the owning thread for that particular resource.
- Deadlocks are sometimes called the “**deadly embrace**.”



How the deadlock occurs

1. Transaction A requests and **acquires a lock** on the invoice_line_items table.
2. Transaction B requests and **acquires a lock** on the invoices table.
3. Transaction A **tries to acquire an lock** on the invoices table to perform the update.
 - Since transaction B already holds a shared lock on this table, transaction A must wait for the exclusive lock.
4. Transaction B **tries to acquire an lock** on the invoice_line_items table, but **must wait** because transaction A holds a shared lock on that table.

Deadlock vs Normal Blocking

Deadlocking is often confused with normal blocking.

- When one transaction has a lock on a resource that another transaction wants, the second transaction waits for the lock to be released.
- By default, PostgreSQL Server transactions **do not time out**.
 - The second transaction is blocked, not deadlocked.
- Blocking can occur on any lock, but blocking exposure is increased when there are many locks lock.
- **Blocking is normal and should be expected.**

Coding techniques that help prevent deadlocks

Don't allow transactions to remain open for very long

- Keep transactions short.
- Keep SELECT statements outside of the transaction except when absolutely necessary.
- Never code requests for user input during an open transaction.

Use the lowest possible transaction isolation level

- The default level of `READ COMMITTED` is almost always sufficient.
- Reserve the use of higher levels for short transactions that make changes to data where integrity is vital.

Coding techniques that help prevent deadlocks (continued)

Make large changes when you can be assured of nearly exclusive access

- If you need to change millions of rows in an active table, don't do so during hours of peak usage.
- If possible, give yourself exclusive access to the database before making large changes.

Consider locking when coding your transactions

- If you need to code two or more transactions that update the same resources, code the updates in the same order in each transaction.